

14장 파일 입출력



학습목표

- 파일과 스트림의 개념을 설명할 수 있다.
- 파일을 열고 닫는 방법을 설명할 수 있다.
- 텍스트 파일 입출력 프로그램을 작성할 수 있다.
- 이진 파일 입출력 프로그램을 작성할 수 있다.
- 임의 접근 입출력 방법을 설명할 수 있다.

파일 이해

C 언어는 파일 입·출력 함수를 이용하여 프로그램이 실행될 때 변수나 배열의 내용을 파일(하드디스크)에 저장하고, 다음 실행 시 파일로부터 데이터를 메모리에 읽어 다시 사용할 수 있음

실행중인 메모리의 특정 내용을 지속적으로 사용하려면



프로그램이 종료 전 디스크의 파일로 저장

파일 이해

● 파일의 개념

- 파일이란 자료를 디스크와 같은 보조 기억장치에 저장하기 위한 특별한 형식
 - 모든 파일은 파일 이름과 확장자를 가지고, 확장자는 파일의 종류를 알려주는 역할을 함
- 파일의 구성
 - 파일은 바이트들이 순차적으로 저장되어 있고 마지막에는 EOF(end of file) 표시가 있음
 - 파일 위치 지시자(file position indicator)가 있어 자료를 읽고 쓸 수 있는 현재의 위치를 나타내고 있음
 - 파일을 열면 위치 지시자는 파일의 첫째 바이트를 가리킴



파일 이해

- 스트림(stream)

- 스트림이란 입출력 장치와 프로그램을 연결하는 채널로 연속된 바이트들의 흐름으로 기본적으로 버퍼가 포함되어 있음
- 스트림을 통한 입출력은 키보드, 모니터, 보조 기억장치 등 입출력 장치의 종류에 관계없이 동일한 방법으로 수행되며
 - 동일한 라이브러리 함수를 사용
 - 파일도 하나의 입출력 장치로 취급
 - 파일 입출력도 키보드, 모니터와 동일한 방법으로 처리할 수 있음



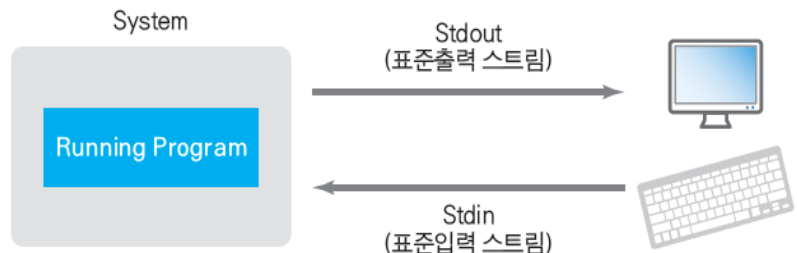
파일 이해

● 스트림(stream)

- 운영체제 에 의해서 자동으로 생성되고 관리되므로, C 개발자가 표준 스트림을 위한 연결과정을 작성할 필요가 없음
- 기본적인 스트림은 자동으로 생성되며, 표준입력 스트림은 stdin이며, 표준출력 스트림은 stdout을 사용
 - 표준 입력 스트림(stdin)
 - 표준 입력으로 작용하며 통상 키보드를 가리킴
 - 표준 출력 스트림(stdout)
 - 표준 출력으로 작용하며 통상 모니터를 가리킴
 - 표준 오류 스트림(stderr)
 - 표준 오류로 작용하며 통상 모니터를 가리킴

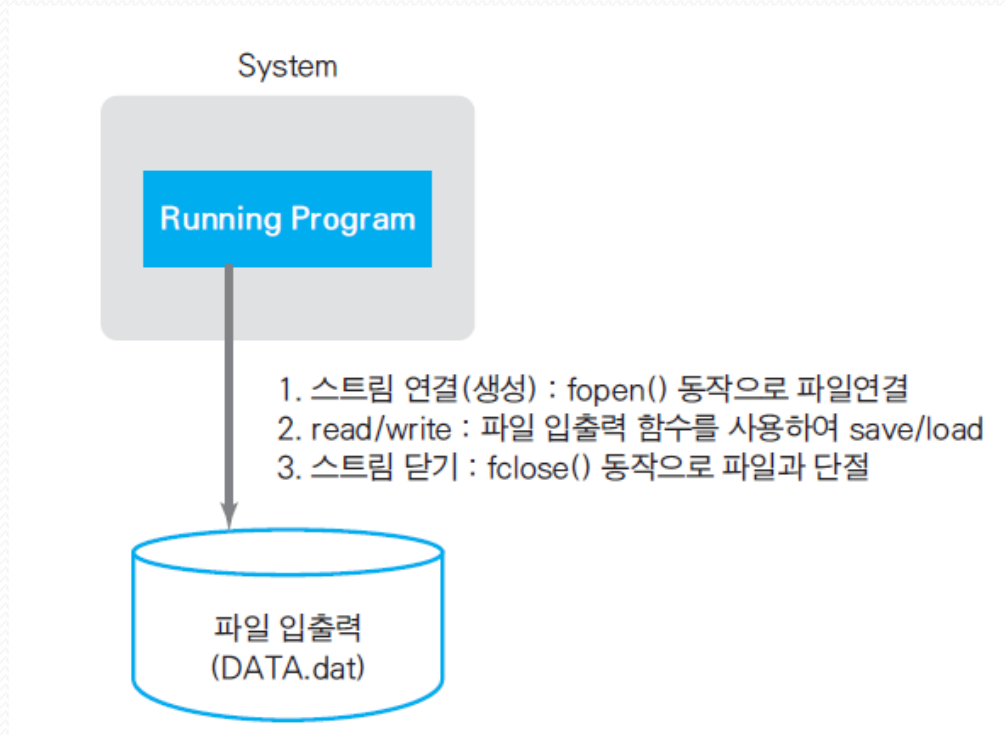
표준 입/출력 스트림

- 표준입출력 장치를 위한 스트림
- 운영체제에 의해 자동 생성 및 관리
→ C 개발자 : 표준 스트림을 위한 연결과정 작성 불필요



파일 이해

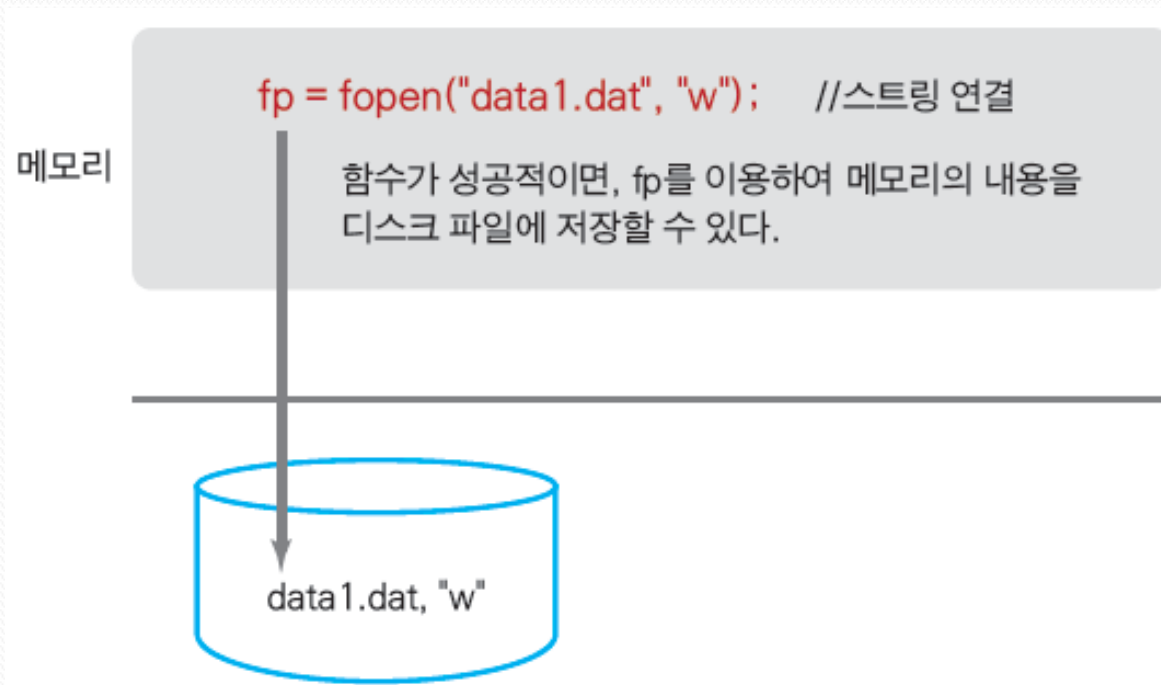
- 파일 입/출력을 위한 스트림
 - 파일을 입출력 하기 위해서는 스트림을 연결해야 함
 - 스트림은 open 동작으로 파일에 연결되고 사용이 끝나면 close 동작 으로 파일과 연결을 단절시킴



파일 열기와 닫기

● 파일 열기

- 파일의 자료를 읽거나 쓰기 위해서는 접근할 파일을 열어야 함
- 파일을 열어 스트림과 연결시키기 위해 `fopen()` 함수를 사용



파일 열기와 닫기

● 파일 열기

- 파일 입출력을 위한 스트림은 파일 구조체(FILE)를 통하여 구현
- 파일의 접근(access)은 파일 포인터(file pointer)를 이용
 - 파일 포인터란 파일의 크기, 현재 위치, 접근 방법 등 파일에 관한 여러 정보를 저장하고 있는 구조체의 포인터를 의미함
- 파일을 열기 위해서는 우선 파일 포인터를 저장할 변수(fp)를 선언
- fopen()과 fopen_s() 함수는 "파일 이름"의 파일을 "모드"에 맞게 연 후 파일의 정보가 들어있는 FILE 구조체의 시작 주소를 &fp에 저장하여 파일 포인터와 연결
- 파일 열기하는 도중 오류가 발생하면(파일 열기가 실패하면) 파일 작업을 할 수 없으므로 반드시 이를 점검해야 함

파일 열기와 닫기

● FILE 구조체

- 파일 구조체는 `stdio.h` 파일에 FILE 구조체형으로 정의
- FILE은 스트림에 접근하기 위한 자료구조
 - 스트림은 `fopen()` 함수가 실행될 때 FILE에 연결할 장치에 대한 정보를 저장
 - 파일의 크기, 현재 위치, 파일의 접근방법 등 파일접근 시 필요한 정보가 저장되는 구조체

스트림은 `fopen()` 함수 실행 시 **FILE**에 연결할 장치에 대한 정보 저장



파일 접근 시 필요한 정보가 저장되는 구조체

파일 열기와 닫기

- 파일 열기

- fopen() 함수를 사용

```
#include <stdio.h>
```

```
FILE *fopen(char *filename, char *openmode);
```

- 설명 : filename으로 지정한 파일을 openmode에 지정한 모드로 열기
- 인수 : filename – 접근할 파일의 이름. 절대 경로와 상대 경로로 지정
openmode – 파일을 어떻게 열 것인지 접근 모드 지정

파일 open 성공

유효한 FILE 포인터 반환

파일 open 실패

유효한 널 포인터(NULL Pointer) 반환

파일 열기와 닫기

- 파일 열기

- C11 표준부터 안전이 강화된 `fopen_s()` 함수를 사용

```
#include <stdio.h>
```

```
errno_t fopen_s(FILE** pFile, const char *filename, const char *mode);
```

- 설명 : filename으로 지정한 파일을 mode에 지정한 모드로 연다.
- 인수 : pFile – 열린 파일에 대한 포인터를 받는

파일 포인터에 대한 포인터

filename – 접근할 파일의 이름. 절대 경로와 상대 경로로 지정

mode – 파일을 어떻게 열 것인지 접근 모드 지정

파일 open 성공

0 반환

파일 open 실패

오류 코드 반환

파일 열기와 닫기

- 파일 열기

- (예) open() 함수로 파일 test.dat을 읽기 전용으로 열기

```
FILE *fp;  
if (( fp = fopen("test.dat", "r") ) == NULL) {  
    printf("File open error \n");  
    exit(1);  
}
```

- (예) fopen_s() 함수로 파일 test.dat을 읽기 전용으로 열기

```
FILE *fp;  
errno_t err;  
err = fopen_s( &fp, "test.dat", "r" );  
if (err == 0)  
    printf("The file 'test.dat' was opened\n");  
else {  
    printf("The file 'test.dat' was not opened\n");  
    exit(1);  
}
```

파일 열기와 닫기

- 파일 열기

- 파일을 열 때는 그 사용 용도를 명시
 - 파일의 접근 모드를 지정
 - 파일에 자료를 저장할 것인가 파일에 있는 자료를 읽어올 것인가 아니면 파일에 어떤 자료를 추가할 것인가 등
- 파일의 형태를 지정
 - 파일의 형태가 텍스트(text) 파일인지 이진(binary) 파일인지도 지정
 - 파일의 형태를 지정하지 않으면 텍스트 파일로 처리

파일 열기와 닫기

- 파일의 형태 : 텍스트 파일
 - 파일 안의 데이터가 오직 텍스트(표시용 문자와 줄바꿈 문자)로만 구성되어 있는 파일로 여러 행으로 구성
 - 줄바꿈(new line) 문자('\n')
 - 다음 행의 시작 지점으로 이동하라는 표시로 운영체제에 따라 다르게 표시
 - 줄바꿈 문자('\n')는 파일에 저장할 때는 CR('\r')과 LF('\n')의 두 문자로 저장하고, 파일에서 읽을 때도 CR('\r')과 LF('\n') 두 문자를 줄바꿈 문자('\n')로 번역하여 다음 행의 시작 지점으로 이동

개행 문자[뉴라인 → 개행] ('\n') 출력 → CR(Carriage Return)과 LF(Linefeed) 문자로 변환



- 스트림에 전달된 내용과 실제 파일에 저장된 내용이 동일하지 않을 수 있음.
- 텍스트 **파일이기 때문에** 우리가 쉽게 저장된 내용을 확인할 수 있음

파일 열기와 닫기

- 파일의 형태 : 이진파일
 - 행 단위가 아닌 이진수 형식으로 된 파일
 - 모든 데이터를 변경하지 않고 파일에 쓰고 읽음
 - 표시할 수 없는 문자들도 포함되어 있어 제대로 표시할 수 없음

2진(Binary) 모드

fopen() 시 mode 는 "rb", "wb", "ab"

- 어떤 종류의 데이터에도 사용 가능
- 문자 변환은 일어나지 않음.

전달된 내용 = 실제 파일에 저장된 내용 동일



- 이진파일은 텍스트파일이 **아니기 때문에** 메모장에서 **확인 안됨**
- 프로그램을 통해 읽혀지는 데이터로만 사용 가능

파일 열기와 닫기

- 파일 열기
 - 파일 열기 모드

모드	처 리	파일이 없는 경우	파일이 있는 경우
"r"	텍스트 파일 읽기(read)	NULL 반환	정상 처리
"w"	텍스트 파일 저장(write) 하기	새로운 파일 생성	이전 내용 삭제
"a"	텍스트 파일 추가(append)	새로운 파일 생성	이전 내용 뒤에 추가
"r+"	텍스트 파일 읽고 저장하기	NULL 반환	정상 처리
"w+"	텍스트 파일 읽고 저장하기	새로운 파일 생성	이전 내용 삭제
"a+"	텍스트 파일 추가로 읽고 저장	새로운 파일 생성	이전 내용 뒤에 추가
"rb"	2진 파일 읽기	NULL 반환	정상처리
"wb"	2진 파일 저장 하기	새로운 파일 생성	이전 내용 삭제
"ab"	2진 파일 추가	새로운 파일 생성	이전의 내용 뒤에 추가
"r+b"	2진 파일 읽고 저장하기	NULL 반환	정상 처리
"w+b"	2진 파일 읽고 저장하기	새로운 파일 생성	이전 내용 삭제
"a+b"	2진 파일 추가로 읽고 저장하기	새로운 파일 생성	이전 내용 뒤에 추가

파일 열기와 닫기

- 파일 열기

- 주어진 파일에 특정 스트림을 열기(다시 열기 위해) 위해 freopen() 함수 사용

➞ 형식

```
FILE *freopen(char const *filename, char const *mode, FILE *stream);
```

➞ 예

```
fp = freopen("data1.dat", "r", fp);
```

이미 열린 스트림을 재 연결하면서 파일모드 변경, 파일 포인터의 현재 위치를 처음으로 보냄.

파일 열기와 닫기

● 파일 닫기

- 더 이상 파일의 읽거나 쓰기가 필요치 않으면 파일을 닫아 줌
- `fclose()` 함수를 사용
 - 현재 열린 파일과 파일 포인터의 연결을 해제
 - 버퍼의 내용을 자동으로 디스크에 쓰고 버퍼를 비움
 - 프로그램 종료 시 열려있는 모든 파일은 자동으로 닫힘

```
#include <stdio.h>
```

```
FILE *fclose(FILE *fp);
```

- 설명 : 열려 있는 파일(fp)를 닫는다.
- 인수 : fp – 파일에 대한 파일 포인터
- 반환 값 : 정상 수행시 - 0
오류 발생시 - EOF, 이는 -1에 해당한다.

파일 열기와 닫기

- 파일 닫기

- (예)

```
FILE *rfp;  
errno_t err;
```

```
err = fopen_s( &rfp, "test.dat", "r" );  
if (err != 0) {  
    printf("The file 'test.dat' was not opened\n");  
    exit(1);  
}
```

```
...
```

```
fclose(rfp);           //fopen() 함수로 정상 수행된 파일 포인터 변수
```

파일 입·출력

● 스트림 입출력 함수

목 적	임의의 스트림	표준 입출력만 사용
문자 입력	fgetc, getc	getchar
문자 출력	fputc, putc	putchar
한 행(문자열) 입력	fgets	gets
한 행(문자열) 출력	fputs	puts
형식화된 입력	fscanf	scanf
형식화된 출력	fprintf	printf
이진 데이터 입력	fread	
이진 데이터 출력	fwrite	




텍스트 스트림 함수 이진 스트림 함수

파일 입·출력

- 현재 위치(Current Location)의 의미
 - 다음 파일 접근이 일어날 파일 내의 위치 정보

현재 위치(Current Location)



스트림이 문자를 쓴(혹은 문자를 읽은) 다음 위치

자동으로 현재 접근할 위치 정보를 FILE 구조체가 가짐.

텍스트 파일 입·출력

- 문자 단위로 파일로부터 입력하는 함수 : fgetc() 함수
 - 읽으려는 스트림으로부터 한 문자를 읽음
 - 반환형은 char형이 아니고 int형을 반환
 - 읽은 문자를 int로 반환하거나 혹은 EOF를 반환하는데, 이 때의 EOF는 오류를 나타내거나 혹은 파일의 끝(end of file)을 나타내는 것으로 -1로 정의되어 있음

```
#include <stdio.h>
```

```
int fgetc(FILE *fp);
```

- 설명 : fp가 가리키는 파일로부터 문자 하나(한 바이트)를 unsigned char 형으로 읽어 정수로 반환한다.
- 인수 : fp – 파일에 대한 파일 포인터
- 반환 값 : 정상 수행시 – 읽은 문자 반환
오류 발생 혹은 파일의 끝 – EOF

텍스트 파일 입·출력

- 문자 단위로 파일로부터 출력하는 함수 : fputc() 함수
 - 한 문자를 스트림에 출력
 - 기록한 문자를 반환하거나 EOF를 반환하는데, 이 때의 EOF는 오류를 나타냄

```
#include <stdio.h>
```

```
int fputc(int ch, FILE *fp);
```

- 설명 : fp가 가리키는 파일에 ch를 unsigned char 형으로 쓴다.
- 인수 : ch – 파일에 쓸 문자
fp – 자료를 쓸 파일의 파일 포인터
- 반환 값 : 정상 수행시 – 출력한 문자 반환
오류 발생시 – EOF

텍스트 파일 입·출력

- 예제 14.1 (ex14_1.c)

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char *s = "나비, 제비야, 깍치지 마라.";
    char c;
    FILE *fp;
    errno_t err;

    err = fopen_s(&fp, "test.dat", "w");
    if (err == 0)
        printf("파일 'test.dat'는 쓰기 위하여 열렸습니다.\n");
    else {
        printf("파일 'test.dat'는 쓰기 위하여 열리지 않았습니다.\n");
        exit(1);
    }
}
```

텍스트 파일 입·출력

- 예제 14.1 (ex14_1.c)

```
while (*s){
    if (fputc(*s++, fp) == EOF) {
        printf("파일쓰기 오류\n");
        exit(1);
    }
}
fclose(fp);

err = fopen_s(&fp, "test.dat", "r");
if (err == 0)
    printf("파일 'test.dat'는 읽기 위하여 열렸습니다.\n");
else {
    printf("파일 'test.dat'는 읽기 위하여 열리지 않았습니다.\n");
    exit(1);
}
```

텍스트 파일 입·출력

- 예제 14.1 (ex14_1.c)

```
while ((c = fgetc(fp)) != EOF)
    putchar(c);
printf("\n");
fclose(fp);
```

```
return 0;
```

```
}
```

```
파일 'test.dat'는 쓰기 위하여 열렸습니다.
파일 'test.dat'는 읽기 위하여 열렸습니다.
나비, 제비야, 갹치지 마라.
계속하려면 아무 키나 누르십시오 . . .
```

텍스트 파일 입·출력

- 예제 14.1.1 (ex14_1_1.c)
 - 예제 14.1의 프로그램을 fopen() 함수로 작성한 프로그램

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char *s = "나비, 제비야, 깎치지 마라.";
    char c;
    FILE *fp;

    fp = fopen("test.dat", "w");
    if (fp == NULL) {
        printf("파일 'test.dat'는 쓰기 위하여 열리지 않았습니다.\n");
        exit(1);
    }
    else
        printf("파일 'test.dat'는 쓰기 위하여 열렸습니다.\n");
}
```

텍스트 파일 입·출력

- 예제 14.1.1 (ex14_1_1.c)
 - 예제 14.1의 프로그램을 fopen() 함수로 작성한 프로그램

```
while (*s){
    if (fputc(*s++, fp) == EOF) {
        printf("파일쓰기 오류\n");
        exit(1);
    }
}
fclose(fp);

fp = fopen("test.dat", "r");
if (fp == NULL) {
    printf("파일 'test.dat'는 읽기 위하여 열리지 않았습니다.\n");
    exit(1);
}
else
    printf("파일 'test.dat'는 읽기 위하여 열렸습니다.\n");
```

텍스트 파일 입·출력

- 예제 14.1.1 (ex14_1_1.c)

- 예제 14.1의 프로그램을 fopen() 함수로 작성한 프로그램

```
while ((c = fgetc(fp)) != EOF)
    putchar(c);
printf("\n");
fclose(fp);

return 0;
```

```
}
```

```
파일 'test.dat'는 쓰기 위하여 열렸습니다.
파일 'test.dat'는 읽기 위하여 열렸습니다.
나비, 제비야, 갓치지 마라.
계속하려면 아무 키나 누르십시오 . . .
```

텍스트 파일 입·출력

● 예제 14.2 (ex14_2.c)

- 텍스트 파일 복사하기(주석 처리 부분은 fopen() 함수를 사용한 코드)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *fp1, *fp2;
```

```
    char c;
```

```
    errno_t err1, err2;
```

```
    err1 = fopen_s(&fp1, "ex14_2.c", "r");    // fp1 = fopen("ex14_2.c", "r");
```

```
    err2 = fopen_s(&fp2, "temp.c", "w");    // fp2 = fopen("temp.c", "w");
```

```
    if (err1 != 0 || err2 != 0) {                // if (fp1 == NULL || fp2 == NULL) {  
        puts("파일1 열기 실패");  
        return 1;  
    }
```

```
}
```

텍스트 파일 입·출력

- 예제 14.2 (ex14_2.c)
 - 텍스트 파일 복사하기 (주석 처리 부분은 fopen() 함수를 사용한 코드)

```
while ((c = fgetc(fp1)) != EOF)
    fputc(c, fp2);
fclose(fp1);
fclose(fp2);
```

```
printf_s("ex14_2.c 파일을 temp.c 파일로 복사 하였습니다.\n");
```

```
return 0;
```

```
}
```

```
ex14_2.c 파일을 temp.c 파일로 복사 하였습니다.
계속하려면 아무 키나 누르십시오 . . .
```


텍스트 파일 입·출력

- 문자열 단위 입·출력
 - 문자열 단위의 입·출력 함수로는 fgets()와 fputs() 함수가 있음
 - gets()와 puts() 함수와 같은 역할을 하는 함수
 - 입출력 장치로부터 문자열을 입·출력하는 함수
 - fgets()와 fputs() 함수
 - 파일로부터 문자열을 입·출력하는 함수
 - fgets() 함수는 gets() 함수와 달리, 읽을 문자열의 최대 개수를 지정하므로 버퍼를 벗어나서 다른 메모리에 영향을 주지 않으므로 fgets() 함수를 그대로 사용하여도 문제가 없음

텍스트 파일 입·출력

- 문자열 단위 입력 : fgets() 함수
 - 스트림에서 문자들을 읽어 버퍼(buffer)로 복사

```
#include <stdio.h>
```

```
char *fgets(char *s, int n, FILE *fp);
```

- 설명 : fp가 가리키는 파일로부터 최대 n-1개의 문자를 읽어 s에 저장한 후 끝에 NULL 문자를 붙인다. 줄 바꿈 문자를 만나거나, 파일의 끝에 도달할 때까지 읽어 들인다.
- 인수 : s – 읽은 문자열을 저장할 변수
n – 읽을 문자열의 최대 개수 (NULL 문자 포함)
fp – 자료를 읽을 파일에 대한 파일 포인터
- 반환 값 : 정상 수행시 – 읽은 문자열 반환
오류 발생 혹은 파일의 끝 – NULL 포인터

텍스트 파일 입·출력

- 문자열 단위 출력 : fputs() 함수
 - fp와 관련된 파일에 버퍼(buffer)가 가리키는 문자열을 출력

```
#include <stdio.h>
```

```
Int fputs(char *s, FILE *fp);
```

- 설명 : NULL이 나올 때까지 문자열 s의 자료를 fp가 가리키는 파일에 쓴다. NULL 문자는 파일에 쓰지 않는다.
- 인수 : s – 파일에 쓸 문자열
fp – 자료를 쓸 파일의 파일 포인터
- 반환 값 : 정상 수행시 – 음수가 아닌 값을 반환
오류 발생시 – EOF

텍스트 파일 입·출력

- 예제 14.3 (ex14_3.c)

```
#include <stdio.h>
#include <stdlib.h>

int main( void )
{
    FILE *fp;
    char line[20] = "Hello! World.";

    if ( fopen_s( &fp, "test.dat", "w" ) != 0 )    // if ( (fp = fopen("test.dat", "w" )) == NULL )
    {
        puts("File open error!");
        exit(1);
    }
    if ( fputs( line, fp ) == EOF )
        puts("fputs() error" );
    else
        fputc('\n', fp);
    puts("파일이 정상적으로 생성되었습니다.");

    fclose( fp );
}
```

텍스트 파일 입·출력

- 예제 14.3 (ex14_3.c)

```
if ( fopen_s( &fp, "test.dat", "r" ) != 0 )      // if ( (fp = fopen("test.dat", "r" )) == NULL )
{
    puts("File open error!");
    exit(1);
}
if ( fgets( line, 20, fp ) == NULL)
    puts("fgets() error" );
else {
    puts("읽은 파일의 내용은 : ");
    printf("%s", line);
}
fclose( fp );
}
```

```
파일이 정상적으로 생성되었습니다.
읽은 파일의 내용은 :
Hello! World.
계속하려면 아무 키나 누르십시오 . . .
```

텍스트 파일 입·출력

- 형식화된 입·출력

- 형식화된 입·출력은 형식 지정자에 대응하는 다양한 자료형을 스트림에 쓰고, 읽어 들일 수 있게 처리
- 형식화 된 입출력의 장점은 텍스트 형식을 갖는 파일에서 여러 가지 다양한 데이터 입·출력 처리가 가능
- fscanf(), fscanf_s(), fprintf(), fprintf_s() 함수가 있음.
 - 입출력 파일을 지정한다는 것 이외에는 표준 입출력 장치로부터 입·출력하는 함수인 scanf(), scanf_s(), printf(), printf_s() 함수와 동일한 기능을 가지고 사용할 수 있음
 - 표준 입·출력장치로부터인가 아니면 파일로부터 입·출력하는 것인가만 다를 뿐임

(예) stdout은 표준 출력 스트림, stdin은 표준 입력 스트림
fprintf(stdout, "Hello!\n"); ≡ printf("Hello!\n");
fscanf(stdin, "%d", &a); ≡ scanf("%d", &a);

텍스트 파일 입·출력

- 형식화된 입력 : fscanf() 함수

- 입력된 문자들을 읽어 와서 이를 format 문자열(형식 문자열)로 지정된 코드에 맞게 변환함
- 입력을 인수로주어진 스트림을 대상으로 데이터를 읽어 들임
- 형식 문자열의 끝에 도달했거나 입력이 지정된 형식 문자열과 일치하지 않을 경우 입력이 종료됨

```
#include <stdio.h>
```

```
int fscanf( FILE *fp, const char *format [, argument] ... );
```

- 설명 : fp가 가리키는 파일로부터 형식이 지정된 데이터를 읽어 인수로 지정한 변수에 가져옴
- 인수 : fp – 자료를 읽을 파일에 대한 파일 포인터
format – scanf() 함수와 동일한 형식
argument – 저장할 변수의 주소
- 반환 값 : 정상 수행시 – **읽어온 바이트 수(입력 값의 개수)**
오류 발생시 – EOF

텍스트 파일 입·출력

- 형식화된 입력 : fscanf_s() 함수
 - C11 표준부터는 안전 문제로 fscanf_s() 함수를 사용

```
#include <stdio.h>
```

```
int fscanf_s( FILE *fp, const char *format [, argument] ... );
```

- 설명 : fp가 가리키는 파일로부터 형식이 지정된 데이터를 읽어 인수로 지정한 변수에 가져온다.
- 인수 : fp – 자료를 읽을 파일에 대한 파일 포인터
format – scanf_s() 함수와 동일한 형식
argument – 저장할 변수의 주소
문자 혹은 문자열인 경우에는 두 번째 인수로 자료가 저장되는 버퍼의 크기를 지정해야 한다.
- 반환 값 : 정상 수행시 – 읽어온 바이트 수
오류 발생시 – EOF

텍스트 파일 입·출력

- 형식화된 출력 : fprintf(), fprintf_s() 함수
 - 인수 목록에 있는 해당 값들의 **출력 형식을 format 인수에 있는 형식 코드 등에 따라 바꿔 스트림으로 출력**
 - fprintf() 함수를 이용하여 만든 파일의 내용을 읽으려면 파일에 쓸 때의 형식과 순서에 정확히 맞게 읽어야 함
 - C11 표준부터는 안전이 강화된 fprintf_s() 함수를 사용할 수 있음
 - fprintf() 함수가 형식 문자열이 NULL 포인터인지 아닌지만 검사하는데 비해 fprintf_s() 함수는 유효한 형식 지정자들에 대한 형식 문자열을 검사한다는 것이고, 두 함수의 인수는 같음

텍스트 파일 입·출력

- 형식화된 출력 : fprintf(), fprintf_s() 함수

```
#include <stdio.h>
```

```
int fprintf(FILE *fp, const char *format [, argument] ... );
```

```
int fprintf_s(FILE *fp, const char *format [, argument] ... );
```

- 설명 : fp가 가리키는 파일에 format에 맞추어 문자들 혹은 값들을 저장

- 인수 : fp – 자료를 쓸 파일의 파일 포인터

 - format – 파일에 출력할 형식 문자열

 - argument – format(형식 문자열)에 따라 여러 인수를 가짐.

 - 형식 문자열에 있는 형식 지정자의 수와 인수의 수는 같음

- 반환 값 : 정상 수행시 – 저장한 바이트 수

 - 오류 발생시 – 음수

텍스트 파일 입·출력

- 예제 14.4 (ex14_4.c)

```
#include <stdio.h>
#include <stdlib.h>

void main(void)
{
    int k1 = 20000, k2;
    double f1 = 625.78, f2;
    char st1[20] = "Welcome!", st2[20];
    FILE *fp;

    printf_s("원래의 데이터: %s, %d, %lf\n", st1, k1, f1);

    if ((fopen_s(&fp, "test.dat", "w")) != 0)    //if ((fp = fopen("test.dat", "w")) == NULL)
    {
        printf("파일 열기 오류\n");
        exit(1);
    }
    fprintf_s(fp, "%20s %10d %10lf\n", st1, k1, f1);
    fclose(fp);
}
```

텍스트 파일 입·출력

- 예제 14.4 (ex14_4.c)

```
if ((fopen_s(&fp, "test.dat", "r")) != 0)    // if ((fp = fopen("test.dat", "r")) == NULL)
{
    printf("파일 열기 오류\n");
    exit(1);
}

// fscanf(fp, "%20s %10d %10lf", st2, &k2, &f2);
fscanf_s(fp, "%20s %10d %10lf", st2, 20, &k2, &f2);
printf_s("파일의 데이터: %s, %d, %lf\n", st2, k2, f2);

fclose(fp);
}
```

```
원래의 데이터: Welcome!, 20000, 625.780000
파일의 데이터: Welcome!, 20000, 625.780000
계속하려면 아무 키나 누르십시오 . . .
```

텍스트 파일 입·출력

- 실습문제 14.1

- 키보드로부터 파일 이름을 입력 받아 그 파일의 내용을 모니터에 출력하는 프로그램을 작성하시오.

텍스트 파일 입·출력

- 실습문제 14.1 (px14_1.c)

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *fp;
```

```
    char fname[20];
```

```
    char c;
```

```
    printf("읽을 파일의 이름을 입력 하시오 : ");
```

```
    scanf_s("%s", fname, 20);
```

```
    if (fopen_s(&fp, fname, "r") != 0) {
```

```
        puts("파일 열기 실패");
```

```
        return 1;
```

```
    }
```

```
        while ((c = fgetc(fp)) != EOF)
            putchar(c);
```

```
        putchar('\n');
```

```
        fclose(fp);
```

```
        return 0;
```

```
    }
```

텍스트 파일 입·출력

- 실습문제 14.1 (px14_1.c)

```
읽을 파일의 이름을 입력 하시오 : px14_1.c
#include <stdio.h>

int main()
{
    FILE *fp;
    char fname[20];
    char c;

    printf("읽을 파일의 이름을 입력 하시오 : ");
    scanf_s("%s", fname, 20);
    if (fopen_s(&fp, fname, "r") != 0) {
        puts("파일 열기 실패");
        return 1;
    }
    while ((c = fgetc(fp)) != EOF)
        putchar(c);
    putchar('\n');
    fclose(fp);

    return 0;
}
계속하려면 아무 키나 누르십시오 . . . _
```

오류 처리 함수

- 파일에 자료를 입·출력할 때 발생하는 오류를 검출할 수 있는 표준 함수
 - 파일을 열 때 지정한 모드와 다른 모드로 파일에 접근한다든가 접근 범위를 벗어난 상황이 발생할 때, 혹은 파일의 끝에 도달했을 때 이를 알아내어 적절한 처리를 해주어야 함
 - feof(), ferror() 함수
 - fgetc() 함수의 반환 값으로는 파일 읽기 오류가 발생했는지 파일의 끝에 도달했는지를 검출할 수 있음
 - strerror() 함수
 - strerror() 함수는 오류 번호에 대한 시스템 오류 메시지의 포인터를 반환해 줌
 - 안전에 더 효과적인 함수로는 strerror_s() 함수가 있음

오류 처리 함수

- 파일에 자료를 입·출력할 때 발생하는 오류를 검출할 수 있는 표준 함수
 - feof() 함수

```
#include <stdio.h>
```

```
int feof(FILE *fp);
```

- 설명 : fp가 가리키는 파일의 끝을 검출한다.
- 인수 : fp – 파일을 가리키는 파일 포인터
- 반환 값 : 파일의 끝을 지났을 경우 - 0이 아닌 값
파일의 끝이 아닌 경우 - 0

오류 처리 함수

- 파일에 자료를 입·출력할 때 발생하는 오류를 검출할 수 있는 표준 함수
 - `ferror()` 함수

```
#include <stdio.h>
```

```
int ferror(FILE *fp);
```

- 설명 : fp가 가리키는 파일의 오류를 검출한다.
- 인수 : fp – 파일을 가리키는 파일 포인터
- 반환 값 : 정상 수행 시 - 0
오류 발생 시 - 0이 아닌 값

오류 처리 함수

- 파일에 자료를 입·출력할 때 발생하는 오류를 검출할 수 있는 표준 함수
 - strerror() 함수

```
#include <string.h>
```

```
char *strerror(int errnum);
```

- 설명 : 오류 번호(errnum)에 대한 시스템 오류 메시지의 포인터를 반환한다.
- 인수 : errnum – 오류 번호
- 반환 값 : 오류 번호 errnum에 대한 오류 메시지 문자열

오류 처리 함수

- 파일에 자료를 입·출력할 때 발생하는 오류를 검출할 수 있는 표준 함수
 - strerror_s() 함수

```
#include <string.h>
```

```
errno_t strerror_s(char *buffer, size_t size, const int errnum);
```

- 설명 : 오류 번호(errnum)에 대한 시스템 오류 메시지 문자열 (buffer)을 매핑 한다.
- 인수 : buffer – 오류 문자열을 보관할 버퍼
size – buffer의 크기
errnum – 오류 번호
- 반환 값 : 성공하면 0, 실패하면 오류 코드

오류 처리 함수

- 예제 14.5 (ex14_5.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *s = "맨드라미 들마꽃에도 인사를 해야지.";
    char c;
    FILE *fp;
    errno_t err;
    char errmsg[80];

    err = fopen_s(&fp, "test.dat", "w");
    if (err != 0) {
        printf("파일 열기 오류\n");
        exit(1);
    }
}
```

오류 처리 함수

```
while (*s){
    if (fputc(*s++, fp) == EOF) {
        printf(" 파일 쓰기 오류\n");
        exit(1);
    }
}
fclose(fp);

err = fopen_s(&fp, "test.dat", "r");
if (err != 0) {
    strerror_s(errmsg, 80, err);
    printf("오류 : %s\n", errmsg);
    exit(1);
}
```

```
while (!feof(fp)) {
    c = fgetc(fp);
    putchar(c);
    if (ferror(fp)) {
        printf("파일 오류\n");
        break;
    }
}
printf("\n");
fclose(fp);

return 0;
}
```

맨드라미 들마꽃에도 인사를 해야지.
계속하려면 아무 키나 누르십시오 . . .

오류 처리 함수

- 예제 14.5.1 (ex14_5_1.c) (1)
 - 예제 14.5의 프로그램을 fopen(), strerror() 함수로 작성한 프로그램

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *s = "맨드라미 들마꽃에도 인사를 해야지.";
    char c;
    FILE *fp;

    fp = fopen("test.dat", "w");
    if (fp == NULL) {
        printf("파일 열기 오류\n");
        exit(1);
    }
}
```

오류 처리 함수

```
while (*s){
    if (fputc(*s++, fp) == EOF) {
        printf(" 파일 쓰기 오류\n");
        exit(1);
    }
}

fclose(fp);

fp = fopen("test.dat", "r");
if (fp == NULL) {
    printf("파일 열기 오류: %s\n", strerror(errno));
    exit(1);
}
```

```
while (!feof(fp)) {
    c = fgetc(fp);
    putchar(c);
    if (ferror(fp)) {
        printf("파일 오류\n");
        break;
    }
}

printf("\n");
fclose(fp);
return 0;
}
```

맨드라미 들마꽃에도 인사를 해야지.
계속하려면 아무 키나 누르십시오 . . .

이진 파일 입·출력

- 이진 파일 입·출력 함수

- 0과 1로 된 이진수 데이터가 직접 저장되는 파일
 - 파일의 내용 중에는 출력할 수 없는 내용도 있으므로 모니터 혹은 프린터로 출력하면 이상하게 나타날 수 있음
 - 이진 파일은 모니터 상에서 직접 확인할 수 없음
- 이진데이터의 이점은 데이터를 입출력 시 숫자 값을 문자열로 변환하는 데 사용되는 오버헤드와 정밀도 손실을 피할 수 있음
- 변환작업이 필요 없어 텍스트 스트림 보다 속도가 빠름

장점	오버헤드와 정밀도 손실을 피할 수 있음.
제한점	이진 데이터여서 프로그램에 읽혀지는 데이터로만 사용 가능.

이진 파일 입·출력

- 이진 파일 입·출력 함수
 - 블록 단위로 파일에 써넣거나 파일로부터 읽을 수 있음

입출력 단위	파일 입력 함수	파일 출력 함수
블록 단위	fread()	fwrite()

이진 파일 입·출력

- 이진 파일 입력 함수 : fread() 함수
 - 스트림과 관련된 파일에서 num개의 개체(size 길이)를 버퍼(buffer)로 읽어 들임

```
#include <stdio.h>
```

```
size_t fread(void *buffer, size_t size, size_t count, FILE *fp);
```

- 설명 : fp가 가리키는 파일로부터 size 바이트의 자료를 count 개
(전체 데이터는 size × count 바이트) 읽어 들여 buffer에 저장한다.
- 인수 : buffer – 읽은 자료를 저장할 메모리 블록의 주소
size – 바이트 단위의 항목의 크기
count – 읽을 항목의 최대 개수
fp – 자료를 읽을 파일의 포인터
- 반환 값 : 성공적으로 읽은 완전히 찬 항목의 개수,
오류가 발생하거나 count에 도달하기 전에 파일의 끝이 되면
count보다 적을 수 있다.

이진 파일 입·출력

- 이진 파일 출력 함수 : fwrite() 함수
 - 버퍼(buffer)에서 num개의 개체(size 길이)를 스트림과 관련된 파일에 저장

```
#include <stdio.h>
```

```
size_t fwrite(const void *buffer, size_t size, size_t count, FILE *fp);
```

- 설명 : size 바이트 길이의 항목 count 개를 (전체 데이터는 $\text{size} \times \text{count}$ 바이트) buffer로부터 fp가 가리키는 출력 파일에 기록한다.
- 인수 : buffer – 읽을 자료가 있는 메모리 블록의 주소
size – 바이트 단위의 항목의 크기
count – 읽을 항목의 최대 개수
fp – 자료를 출력할 파일의 포인터
- 반환 값 : 성공적으로 기록된 완전히 찬 항목의 개수, 오류가 발생하면 count보다 적을 수 있다.

이진 파일 입·출력

- 예제 14.6 (ex14_6.c)

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    FILE *fp;
    int a[5] = {1, 2, 3, 4, 5};
    int i, b[5];

    if ((fopen_s(&fp, "test.dat", "wb")) != 0) {
// if ((fp = fopen("test.dat", "wb")) == NULL) {
        puts("쓰기 위해 파일 열기 실패");
        exit(1);
    }
    if (fwrite(a, sizeof(int), 5, fp) != 5) {
        puts("파일에 기록 오류");
        exit(1);
    }
    fclose(fp);
}
```

이진 파일 입·출력

```
if ((fopen_s(&fp, "test.dat", "rb")) != 0) {  
// if ((fp = fopen("test.dat", "rb")) == NULL) {  
    puts("읽기 위해 파일 열기 실패");  
    exit(1);  
}  
if (fread(b, sizeof(int), 5, fp) != 5) {  
    puts("파일로부터 읽기 오류");  
    exit(1);  
}  
fclose(fp);  
  
for (i=0; i<5; i++)  
    printf("%d ", b[i]);  
printf("\n");  
  
return 0;  
}
```

1 2 3 4 5
계속하려면 아무 키나 누르십시오 . . .

이진 파일 입·출력

- 예제 14.7 (ex14_7.c)

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *sfp, *dfp;
```

```
    char sfile[20] = {"ex14_7.c"};
```

```
    char dfile[20] = {"temp.c"};
```

```
    char buf[256];
```

```
    int rcnt, wcnt;
```

```
    if ((fopen_s(&sfp, sfile, "rb") != 0) || (fopen_s(&dfp, dfile, "wb") != 0)) {
```

```
//    if (((sfp = fopen(sfile, "rb")) == NULL) || ((dfp = fopen(dfile, "wb")) == NULL)) {
```

```
        fprintf(stderr, "파일 열기 실패...\n");
```

```
        return 1;
```

```
    }
```

이진 파일 입·출력

```
while ((rcnt = fread(buf, 1, sizeof(buf), sfp)) > 0) {  
    wcnt = fwrite(buf, 1, rcnt, dfp);  
    if (wcnt < 0) {  
        fprintf(stderr, "파일 쓰기 오류...\n");  
        return 1;  
    }  
}  
printf("%s에서 %s로 복사 되었습니다\n", sfile, dfile);  
fclose(sfp);  
fclose(dfp);  
  
return 0;  
}
```

ex14_7.c에서 temp.c로 복사 되었습니다
계속하려면 아무 키나 누르십시오 . . .

파일 임의 접근 방법

- 파일 입·출력 방식

- 순차 접근(sequential access) 방식
 - 열려진 파일의 처음에서 끝까지 차례대로 자료를 입·출력
 - 순차 접근 방식에서는 한번 읽은 데이터를 다시 읽으려면 현재의 파일을 닫고 다시 열어야 함
- 임의 접근(random access) 방식
 - 파일의 임의의 위치에서 자료를 입출력
 - 파일 구조체에는 파일의 현재 위치를 가리키는 파일 위치 지시자(file position indicator)라고 하는 포인터 멤버가 가리키는 곳에서부터 읽기와 쓰기가 시작됨
 - 임의 접근 함수는 이진 모드의 파일에서만 정확한 위치로 이동할 수 있다. 텍스트 모드인 경우 입·출력할 때 자료의 변환이 일어나기 때문에 정확한 위치를 찾아가지 못할 수도 있음

파일 임의 접근 방법

- 파일 위치 변경 : fseek() 함수
 - 파일의 임의의 위치로 이동할 수 있도록 하는 함수
 - fseek() 함수는 파일 위치 지시자를 origin으로부터 offset 바이트만큼의 위치로 이동

```
#include <stdio.h>
```

```
int fseek(FILE *fp, long offset, int origin);
```

- 설명 : 파일 위치 지시자를 지정한 위치로 이동한다.
origin으로부터 offset 바이트만큼의 위치로 이동한다.
- 인수 : fp – 파일 구조체의 포인터
offset – origin으로부터의 바이트 수
origin – 기준 위치
- 반환 값 : 정상 수행 시 - 0을 반환한다.
오류 발생 시 - 0이 아닌 값을 반환한다.

파일 임의 접근 방법

- 파일 위치 변경 : `fseek()` 함수
 - 파일 기준 위치 `origin`의 값을 나타내는 매크로

매크로	값	의미
<code>SEEK_SET</code>	0	파일의 시작 위치
<code>SEEK_CUR</code>	1	파일의 현재 위치
<code>SEEK_END</code>	2	파일의 끝

(예)

```
fseek(fp, 0, SEEK_SET);  
    // 파일의 시작에서 0 바이트 위치 즉, 처음으로 접근 위치를 이동  
fseek(fp, 100, SEEK_CUR);  
    // 현재 위치에서 진행(순) 방향으로 100 바이트로 접근 위치를 이동  
fseek(fp, -100, SEEK_END);  
    // 파일의 끝에서 반대(역) 방향으로 100 바이트로 접근 위치를 이동
```

파일 임의 접근 방법

- 파일 현재 위치 정보 확인 : `ftell()` 함수
 - 다음 읽거나 쓰기를 할 위치
 - 파일의 시작부터 현재 위치까지의 바이트 수(오프셋)를 반환

```
#include <stdio.h>
```

```
long ftell(FILE *fp);
```

- 설명 : 파일 포인터 `fp`에 연결된 파일의 읽기와 쓰기의 현재 위치를 구한다.
- 인수 : `fp` – 파일 구조체의 포인터
- 반환 값 : 정상 수행 시 – 현재 파일의 위치의 바이트 `offset`을 반환
오류 발생 시 – `-1L`을 반환한다.

파일 임의 접근 방법

- 파일 위치 정보 되돌리기 : rewind() 함수
 - 파일의 시작 위치로 이동

```
#include <stdio.h>
```

```
void rewind(FILE *fp);
```

- 설명 : 파일 포인터 fp에 연결된 파일의 읽기와 쓰기의 위치를 파일의 시작 위치로 되돌린다.
- 인수 : fp – 파일 구조체의 포인터

파일 임의 접근 방법

- 파일 입·출력 기타 함수

```
int feof( FILE *stream);  
int ferror( FILE *stream);
```

- **feof()** 함수는 현재 스트림이 파일의 끝(EOF)이라면 참을 반환하는 함수다. EOF는 파일의 끝일 때에도 발생하지만, 파일 접근 시 오류가 있을 때에도 발생한다. 이를 구별하기 위해 사용될 수 있다.
- **ferror()** 함수는 이전 내용처럼 EOF 발생시 오류로 인해 발생했는지 확인하는 함수이다. 오류가 발생하면 참을 반환한다.

파일 임의 접근 방법

● 파일 입·출력 기타 함수

```
int remove( char const *filename);  
int rename( char const *oldname , char const *newname);  
int fflush(FILE *stream);
```

- **remove()** 함수는 지정된 파일을 삭제한다. remove()가 호출되었을 때 파일이 열려 있다면, 컴파일러에 따라 다른 결과를 나타낼 수 있다. 따라서 파일을 닫고 호출해야 한다.
- **rename()** 함수는 oldname에서 newname으로 파일 이름을 변경할 때 사용한다. 만일 새 파일 이름이 이미 존재하면 컴파일러에 따라 다른 결과를 나타낼 수 있다. 이 함수가 실패하면 원래의 파일에 여전히 접근할 수 있다.
- **fflush()** 함수는 대상이 되는 파일 버퍼를 비운다. 스트림이 파일 스트림인 경우 파일 버퍼를 비움으로서 버퍼의 내용을 최종적으로 디스크에 기록한다. 또한 스트림이 stdin인 표준 입력일 경우 표준 입력 버퍼를 깨끗이 비운다. 때문에 키보드로부터 입력된 데이터를 모두 버리고자 할 때 사용될 수 있다.

파일 임의 접근 방법

- 예제 14.8 (ex14_8.c)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Nobel_Literature {
    char title[41];
    char author[41];
    unsigned int year;
} NOBEL;

#define RECLEN 86
#define FORMAT "%-40s%-40s%6d"
FILE *fp;
void datawrite(int, char *, char *, int);
void dataread(NOBEL *);
```


파일 임의 접근 방법

- 예제 14.8 (ex14_8.c)

```
void main()
{
    int i;
    NOBEL nbook;

    if ((fopen_s(&fp, "booklist.dat", "wb")) != 0) {
//    if ((fp = fopen("booklist.dat", "wb")) == NULL) {
        puts("파일 쓰기 에러");
        exit(1);
    }
    datawrite(0, "염소의 축제", "마리오 바르가스 요사", 2010);
    datawrite(1, "기억이 나를 본다", "토마스 트란스트뢰메르", 2011);
    datawrite(2, "사부님은 갈수록 유머러스해진다", "모옌", 2012);
    fclose(fp);
}
```

파일 임의 접근 방법

- 예제 14.8 (ex14_8.c)

```
if ((fopen_s(&fp, "booklist.dat", "rb")) != 0) {  
// if ((fp = fopen("booklist.dat", "rb")) == NULL) {  
    puts("파일 읽기 에러");  
    exit(1);  
}  
  
for (i=0; i<3; i++) {  
    dataread(&nbook);  
    printf_s("%d\t제목 : %40s\n\t작가 : %20s \n\t수상년도 : %d \n\n",  
            i, nbook.title, nbook.author, nbook.year);  
}  
}  
  
void datawrite(int recno, char *title, char *author, int year)  
{  
    fseek(fp, RECLEN *recno, SEEK_SET);  
    fprintf_s(fp, FORMAT, title, author, year);  
}
```

파일 임의 접근 방법

- 예제 14.8 (ex14_8.c)

```
void dataread(NOBEL *nbook)
{
    fgets(nbook->title, 41, fp);
    fgets(nbook->author, 41, fp);
    fscanf_s(fp, "%d", &nbook->year);
}
```

```
0      제목 = 염소의 축제
      작가 = 마리오 바르가스 요사
      년도 = 2010

1      제목 = 기억이 나를 본다
      작가 = 토마스 트란스트뢰메르
      년도 = 2011

2      제목 = 사부님은 갈수록 유머러스해진다
      작가 = 모옌
      년도 = 2012

계속하려면 아무 키나 누르십시오 . . .
```

파일 임의 접근 방법

- 실습문제 14.2

- 예제 14.8의 프로그램을 원하는 레코드 번호를 입력 받아 그 레코드의 내용을 읽어 출력하는 형태로 바꾸어 프로그램을 작성하시오.

dataread() 함수의 원형 :

```
void dataread(int rno, NOBEL *nbook);
```

파일 임의 접근 방법

- 실습문제 14.2 (px14_2.c)

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct Nobel_Literature {
    char title[41];
    char author[41];
    unsigned int year;
} NOBEL;
```

```
#define RECLEN 86
#define FORMAT "%-40s%-40s%6d"
```

```
FILE *fp;
void datawrite(int, char *, char *, int);
void dataread(int, NOBEL *);
```

파일 임의 접근 방법

- 실습문제 14.2 (px14_2.c)

```
void main() {  
    int rno;   NOBEL nbook;  
    if ((fopen_s(&fp, "booklist.dat", "wb")) != 0) {  
        puts("파일 쓰기 에러");  
        exit(1);  
    }  
    datawrite(0, "염소의 축제", "마리오 바르가스 요사", 2010);  
    datawrite(1, "기억이 나를 본다", "토마스 트란스트뢰메르", 2011);  
    datawrite(2, "사부님은 갈수록 유머러스해진다", "모옌", 2012);  
    fclose(fp);  
  
    if ((fopen_s(&fp, "booklist.dat", "rb")) != 0) {  
        puts("파일 읽기 에러");  
        exit(1);  
    }  
    printf("출력할 레코드 번호를 입력 하세요: ");  
    scanf_s("%d", &rno);  
    dataread(rno, &nbook);  
}
```

파일 임의 접근 방법

- 실습문제 14.2 (px14_2.c)

```
void datawrite(int recno, char *title, char *author, int year)
{
    fseek(fp, RECLEN *recno, SEEK_SET);
    fprintf_s(fp, FORMAT, title, author, year);
}
```

```
void dataread(int recno, NOBEL *nbook)
{
    fseek(fp, recno * RECLEN, SEEK_SET);
    fgets(nbook->title, 41, fp);
    fgets(nbook->author, 41, fp);
    fscanf_s(fp, "%d", &nbook->year);
    printf_s("%d\t제목 : %40s\n\t작가 : %20s \n\t수상년도 : %d \n\n",
        recno, nbook->title, nbook->author, nbook->year);
}
```

```
출력할 레코드 번호를 입력 하세요: 1
1      제목 : 기억이 나를 본다
      작가 : 토마스 트란스트뢰메르
      수상년도 : 2011
```

```
계속하려면 아무 키나 누르십시오 . . .
```

파일 임의 접근 방법

- 예제 14.9 (ex14_9.c)
 - 이진 파일을 읽어서 출력하는 예

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    FILE *fp;
    char *buf;
    int fsize;

    if ((fopen_s(&fp, "ex14_9.c", "rb")) != 0) {
// if ((fp = fopen("ex14_9.c", "rb")) == NULL) {
        fprintf(stderr, "파일 열기 실패...\n");
        return 1;
    }

    fseek(fp, 0, SEEK_END);
    fsize = ftell(fp);
    buf = (char *)malloc(fsize * sizeof(char) + 1);
```


파일 임의 접근 방법

- 예제 14.9 (ex14_9.c)

```
rewind(fp);
if (fread(buf, 1, fsize, fp) < 1) {
    fprintf(stderr, "파일 읽기 오류...%n");
    return 1;
}
buf[fsize] = '\0';
fclose(fp);

puts("다음은 읽은 파일의 내용입니다.");
puts(buf);

return 0;
}
```

파일 임의 접근 방법

● 예제 14.9 (ex14_9.c)

다음은 읽은 파일의 내용입니다.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *fp;
    char *buf;
    int fsize;

    if ((fopen_s(&fp, "ex14_9.c", "rb")) != 0) {
// if ((fp = fopen("ex14_9.c", "rb")) == NULL) {
        fprintf(stderr, "파일 열기 실패...\n");
        return 1;
    }

    fseek(fp, 0, SEEK_END);
    fsize = ftell(fp);
    buf = (char *)malloc(fsize * sizeof(char) + 1);

    rewind(fp);
    if (fread(buf, 1, fsize, fp) < 1) {
        fprintf(stderr, "파일 읽기 오류...\n");
        return 1;
    }
    buf[fsize] = '\0';
    fclose(fp);

    puts("다음은 읽은 파일의 내용입니다.");
    puts(buf);

    return 0;
}
계속하려면 아무 키나 누르십시오 . . . _
```

Question & Answer

